Multi-Agent Orchestration Framework

WEB INTEGRATION & CUMULATIVE SHARED MEMORY

PYTHON | LANGCHAIN | FASTAPI

Presented by: Anugraha P A
Internal Mentor: MR.Devendar Pratap
Organization: Vidz ai Digital Private Limited **Training Partner:** Infosys Springboard

© 2026 Vidzai Digital | Confidential

vidz.ai

Infosys Springboard

INTRODUCTION

- **Project Goal:** To create a seamless "Human-in-the-Loop" (HITL) AI system.
- **Core Tech Stack:** Python, LangChain, FastAPI, and HTML/JavaScript.
- **Licensing:** Released under the **MIT License** for open-source extensibility

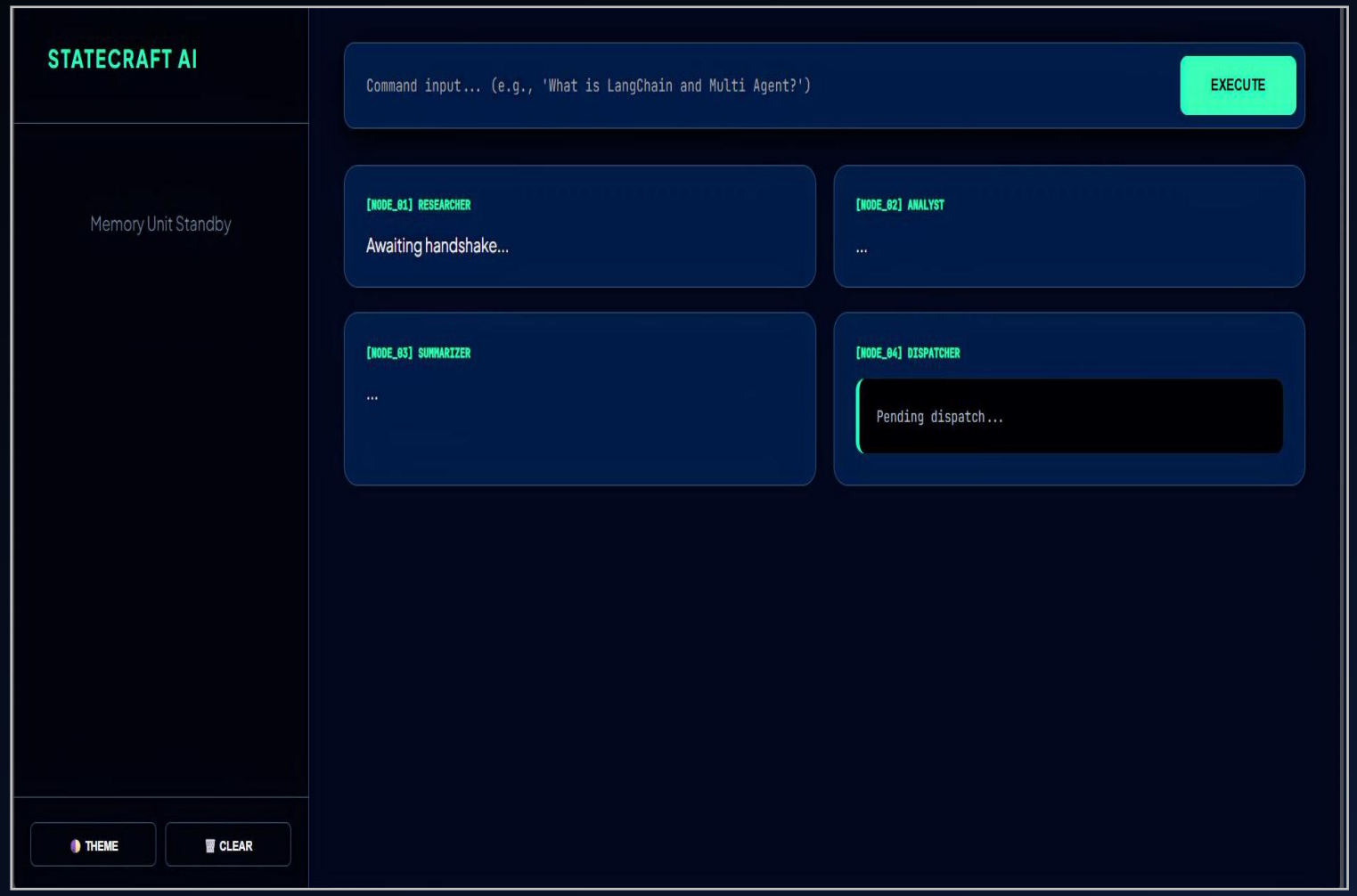
© 2026 Vidzai Digital | Confidential

PROJECT OVERVIEW

- **Architecture**: Uses a Shared Memory Buffer to ensure data persists across the entire conversation.

- **User Interface**: A responsive web dashboard replacing the traditional terminal-based interaction.

- **Concept**: A sequential agentic workflow where specialized agents (Research, Analysis, Summary) collaborate.



© 2026 Vidzai Digital | Confidential

The Milestone Journey

- **M1 & M2**: Basic Agent Logic & Role-playing personas.

User Input → Research Agent → Analysis Agent → Summary Agent → Shared Memory → FastAPI UI.

- **M3**: Implementation of **Shared Memory** to prevent
- "AI Amnesia."

- **M4**: **Full-Stack Integration**—building the FastAPI backend and Frontend UI.

© 2026 Vidzai Digital | Confidential

Key Use Cases

- **AI Research Assistant:** Automating complex technical lookups.
- **Professional Communication:** Drafting context-aware emails with human approval.
- **Knowledge Management:** Building a persistent session- based expert for corporate teams.

© 2026 Vidzai Digital | Confidential

What’s New in This Project?

- **Stateful Sessions:** Memory is cumulative; the AI remembers your first question even during the tenth question.
- **Action Gatekeeping:** Implementation of an "Approval UI" where the agent asks permission before "sending" an email.
- **Asynchronous Processing:** The UI updates in real-time as each agent finishes its specific task.

© 2026 Vidzai Digital | Confidential

Backend Orchestration: LangChain & Shared Memory

```python
# ----------------------------
# Shared Memory
# ----------------------------
shared_memory = []

# ----------------------------
```

•Autonomous Decision Making:
Implemented an AgentExecutor to select tools based on user intent.

```python
    # Store in shared memory
    if not is_duplicate(result):
        shared_memory.append(result)

    return result
```

•Contextual Persistence:

- Integrated ConversationBufferMemory to maintain "state" across the FastAPI web session.

```python
# ---------------- LangChain Tools ----------------
tools = [
    Tool(
        name="Calculator",
        func=calculator_tool,
        description="Use this tool for mathematical calculations."
    ),
    Tool(
        name="WeatherAPI",
        func=weather_tool,
        description="Use this tool to get weather of a city."
    ),
    Tool(
        name="CodingAssistant",
        func=coding_tool,
        description="Use this tool for programming questions."
    ),
    Tool(
        name="KnowledgeBase",
        func=knowledge_tool,
        description="Use this tool for general knowledge questions."
    )
]

print("LangChain Tool Agent Ready (type 'exit' to stop)")
```

•Scalable Integration:
Connected the LangChain logic to a RESTful API for real-time frontend interaction.

© 2026 Vidzai Digital | Confidential

Pros & Cons of New Features

Feature	Pros	Cons
Cumulative Memory	Better context & personalization.	Increased token usage over very long sessions.
FastAPI Backend	High performance & easy to scale.	Requires specific port management (8000).
HITL Approval	High security & reliability.	Slower workflow due to manual intervention.

© 2026 Vidzai Digital | Confidential

Challenges and Solutions

1.Challenge: Syncing Git history and fixing commit labels.

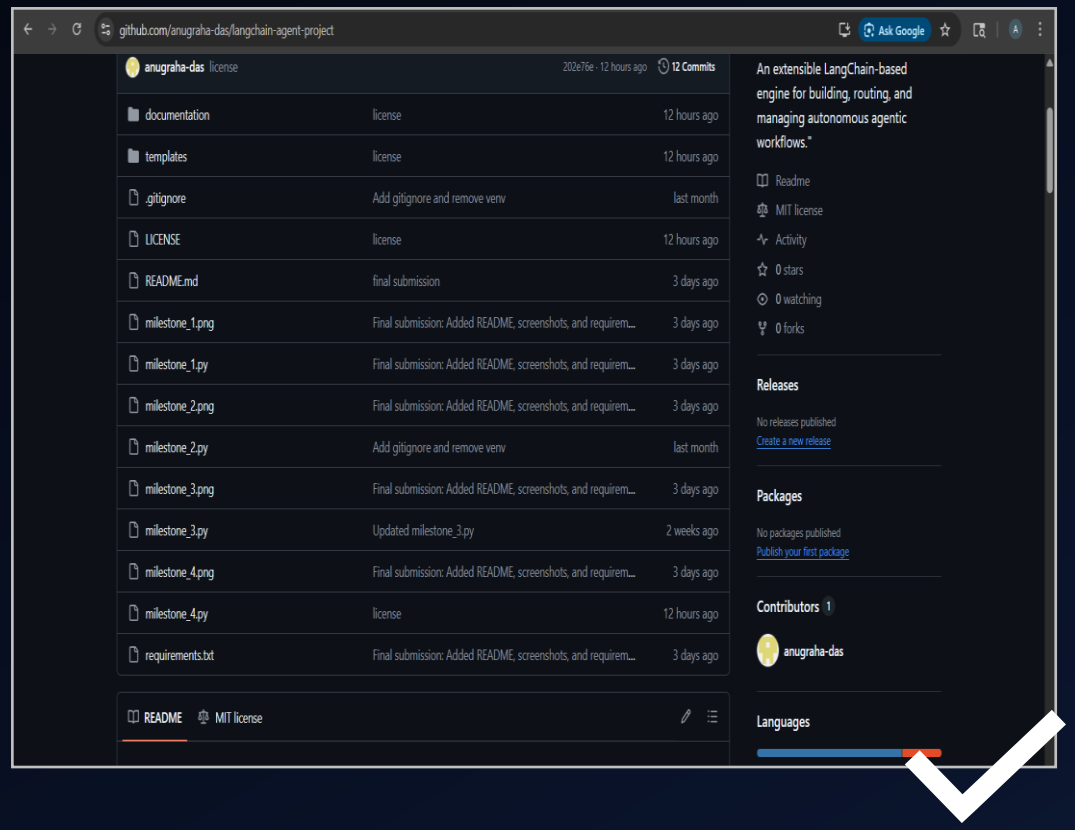
Solution: Used git commit --amend and standardized the **MIT LICENSE** naming to meet GitHub standards.

2.Challenge: "Non-fast-forward" push errors.

Solution: Mastered the git pull --rebase workflow to sync local and remote repositories.

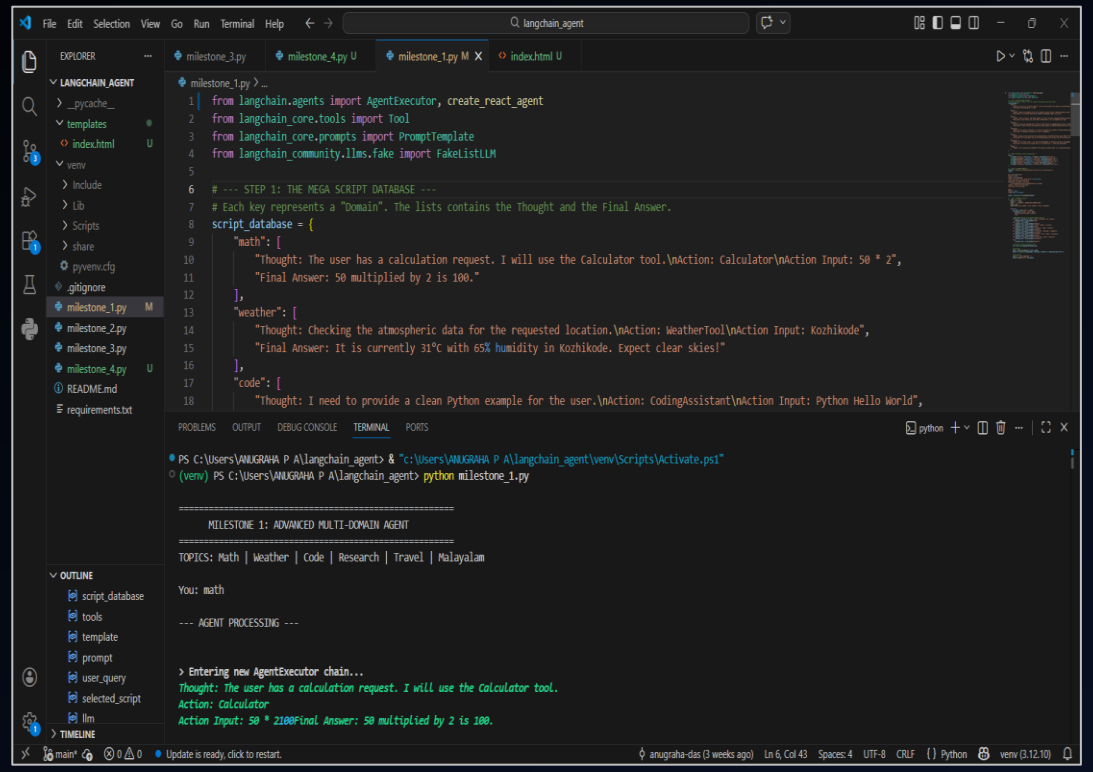
3.Challenge: Handling Excel files in Git.

Solution: Established a dedicated documentation/ folder with proper tracking for Unit Tests and Defect Logs.

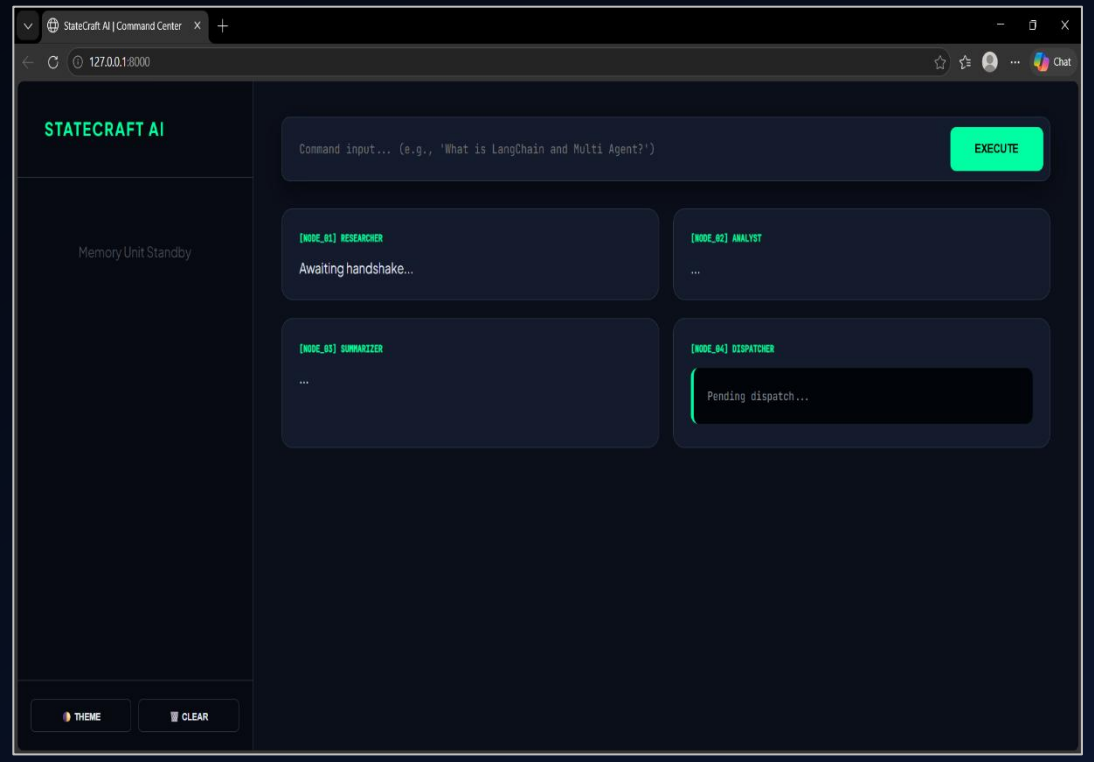


© 2026 Vidzai Digital | Confidential

Before vs. After



Standard Linear Chat (No Context)



"Intelligent Multi-Agent Workflow (Milestone 4)."

"From Developer Tool to Business Solution."

© 2026 Vidzai Digital | Confidential

Future Scope

- **Phase 1 (Current):** Web-based Multi-Agent Orchestration with Shared Memory.
- **Phase 2 (Short-term):** Integration with **Vector Databases (ChromaDB)** for long-term document retrieval.
- **Phase 3 (Long-term):** Enterprise Deployment with **Docker/Kubernetes** and multi-user authentication

© 2026 Vidzai Digital | Confidential

Conclusion

- Successfully transitioned a complex CLI tool into a user-friendly Web Application.
- Demonstrated the power of **LangChain Orchestration** in a real-world business context.
- **Final Project Status:** 100% Completed, Documented, and Licensed.

PYTHON | LANGCHAIN | FASTAPI | MS EXCEL | GITHUB

"Ready for Technical Review and Deployment."

© 2026 Vidzai Digital | Confidential

vidz.ai

Infosys Springboard